Communication entre micro-services
2026

1 Objectifs

L'objectif de ce TP est de réaliser une première application composée de deux microservices Python communiquant par une API REST. À la fin du TP, vous serez capables de :

- créer un microservice Python simple ;
- exposer des fonctionnalités avec une API REST ;
- appeler une API REST depuis un autre microservice ;
- créer une image Docker pour un microservice ;
- exécuter plusieurs microservices dans des conteneurs Docker ;
- faire communiquer les conteneurs au moyen d'un réseau Docker.

1.1 Architecture de l'application

- L'application est composée de deux microservices :
  1. Le microservice `clients` est responsable des informations relatives aux clients.
  2. Le microservice `commandes` est responsable des commandes.
  3. Le microservice `commandes` doit pouvoir interroger le microservice `clients` afin de vérifier qu'un client existe.

1.2 Création du projet

Créer l'arborescence suivante :

```
tp-microservices/
|
+-- clients/
|   +-- app.py
|   +-- requirements.txt
|   +-- Dockerfile
|
+-- commandes/
+-- app.py
+-- requirements.txt
+-- Dockerfile
```

Dans le répertoire `clients`, créer un fichier `requirements.txt` contenant la ou les dépendance(s) nécessaire(s) :

1.3 API REST du microservice Clients

— Créer un serveur Flask écoutant sur le port `8080`. Le microservice doit gérer les clients suivants :

```
clients = [
{"id": 1, "nom": "Alice", "pays": "France"},
{"id": 2, "nom": "Alex",   "pays": "Belgique"},
{"id": 3, "nom": "Madeleine",  "pays": "France"}
]
```

— Créer les endpoints suivants :

Méthode	URL	Fonction
GET	`/clients`	Retourner tous les clients
GET	`/clients/<id>`	Retourner un client

— Tester le serveur localement avec :

```
curl http://localhost:8080/clients
```

Puis :

```
curl http://localhost:8080/clients/1
```

1.4 API REST du microservice Commandes

— Le deuxième microservice gère les commandes.

— Créer les commandes suivantes :

```
commandes = [
{"id": 1, "client_id": 1, "produit": "PC", "montant": 1200},
{"id": 2, "client_id": 2, "produit": "Livre", "montant": 30},
{"id": 3, "client_id": 1, "produit": "Casque", "montant": 80}
]
```

— Le microservice doit exposer :

Méthode	URL	Fonction
GET	`/commandes`	Retourner toutes les commandes
GET	`/commandes/<id>`	Retourner une commande

— Le serveur doit également écouter sur le port `8080`.

1.5 Communication entre les microservices

— Modifier le microservice `commandes` afin qu'il puisse communiquer avec le microservice `clients`.

  — Requête : GET /commandes/1

  — Le microservice `commandes` doit :

    1. récupérer la commande numéro 1 ;
    2. récupérer son `client_id` ;
    3. appeler le microservice `clients` avec une requête HTTP ;

4. récupérer les informations du client ;
5. retourner une réponse contenant la commande et les informations du client.

— Par exemple, la réponse pourra être :

```
{
"id": 1,
"produit": "PC",
"montant": 1200,
"client": {
"id": 1,
"nom": "Alice",
"pays": "France"
}
}
```

— Pour effectuer l'appel HTTP depuis Python, utiliser la bibliothèque `requests`. Ajouter dans `requirements.txt` :

```
flask
requests
```

2 Réseau Docker

2.1 Dockeriser le microservice Clients

— Créer le fichier `Dockerfile` :

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 8080

CMD ["python", "app.py"]
```

— Construire l'image :

```
docker build -t microservice-clients ./clients
```

— Lancer le conteneur :

```
docker run -p 8080:8080 --name clients microservice-clients
```

— Tester :

```
curl http://localhost:8080/clients
```

De manière similaire, dockeriser le microservice commandes. Mettez en place une architecture Docker permettant au microservice Commandes de communiquer avec le microservice Clients par REST, tout en permettant à un utilisateur d'accéder aux deux services depuis l'extérieur. Vous devez déterminer vous-mêmes le réseau Docker à créer, la configuration des conteneurs, les ports à exposer et la manière dont Commandes doit adresser Clients.

2.2 Gestion des erreurs

— Modifier le programme afin que :

  — une commande inexistante retourne le code HTTP `404` ;
  — un client inexistant retourne le code HTTP `404` ;
  — le microservice `commandes` gère le cas où le microservice `clients` n'est pas disponible.

— Tester notamment :

```
curl http://localhost:8080/clients/100
```

et :

```
curl http://localhost:8081/commandes/100
```

3 Travail demandé

À la fin du TP, vous devez fournir :

— le code source des deux microservices ;
— les deux fichiers `Dockerfile` ;
— les deux fichiers `requirements.txt` ;
— les commandes Docker utilisées ;
— une capture montrant les deux conteneurs en fonctionnement ;
— une capture montrant un appel REST de `commandes` vers `clients`.